Python

Fundamentals:

Introduction: It is a high level, general purpose

Syntax is designed to easier for humans to understand than low-level instruction

how Python run?

Print("Hello") code

↓

Python Implementation/runtime

(Interprets and executes the program instruction)

↓

Output

Interpreted language, here source code compiled into bytecode .py → extension

uses Indentation: to identify code blocks

is Case sensitive Ex: name and Name are different variable

Variables in Python → Dynamically typed

int, float, str, bool

↓ Decimal ↓ Text ↓ True or false

name = "test", age = 33. print(typeOf(name))

↓ 'str'

no need to declare explicity

Casting:

int(), float(), str(), bool()

Ex: age = "21" print(age + "2") → 212

age = int(age) . print(age + 2) → 23

↓ cast into int

Operators	Java	Python
Logical and	&&	and
Logical or	|	-
Logical not	!	not
Exponential	Math.pow(a,b)	a**b
Floor Division	(int)(a/b)	a//b → nearest whole number 7//2 → 3
String Comparison	.equals()	(==) → compare structural value
Obj identity	==	(is) / is not → check if two variables point to exact same memory
Membership check	req Collection method	in/not in ex: "a" in "apple" → Check if an item exist in collection

a/b → perform division return float

a//b → perform floor division, %→ gives remainder

Input and Output:

input() → asks user question

Print() → display the result

Ex: name = input("Name: ")
print(name):

~~a = input(~~ a = int(input("Enter number: ")

String formatting → uses (f)

name = "Arun"

age = 21

print (f"My name is {name} and age is {age}")

formatting decimal values

Price = 99.5678

Print(f"Price: {price:.2f}")

Conditional Statements

Python relies on strict indentation

if statement, if else,

if - elif - else

↳ (else if) in python

Ex:

```
if (condition):
    //statement
else :
```

Truthy falsy values

empty and zero values are falsy (None, 0, 0.0, false, "", (), {}, [])

others non-zero and non empty string are truthy

Looping:

For loop: Ex: for i in range(5) → no. of iterations

range (start, stop, step) (1, 3, 5, 7, 9)

range(1, 10, 2): →

enumerate() items = ["a", "b", "c"]

↳ for index, value in enumerate(items)

print(f"{index}, {value}")

while loop: while(condition):

④ Both for and while loop support else block

else block after completion of loop

break → stop loop

Continue → skip

pass → Do nothing

Mutable vs Immutable

Mutable	Immutable
list	int
dict	float
set	bool
custom class instance	str
	tuple*

* structure can not be change, but it can contain mutable obj whose content can change

Data structures → give us ways to store, access, organize and transform data

Structure	Main Purpose
List → ordered, mutable collection of items	sequential data, Stack, queue

Do not need to be same datatype
0-based indexing
fruits = ["apple", "banana", "Guava"]
extend() len() → length

methods:
append(), ↗ index
insert(—, —)
remove(5), value
pop() → remove last

Tuple → An ordered collection that cannot have its element, reassigned, added or removed (immutable)
faster than list
Ex: ↗ store multiple values in single variable
user = ("Alice", 20, True)

constant data, coordinates, return value

can contain list, which can still change
record = ("Alice", [45, 50])
record[1].append(95)
Print(record) => 45, 50, 95

Set → Set stores unique elements, do not guarantee meaningful order (unordered)
id = {101, 102, 103}

methods ↓
add() → single element
update() → multiple
remove()

Set Operations:

union	S1.union(S2)	S1|S2 All unique from both
Intersection	S1.intersection(S2)	S1&S2 common to both
difference	S1.difference(S2)	S1-S2 items S1 not in S2
Symmetric difference		S1^S2 items in either set not & both

Set comparisons → return boolean by comparing 2 sets

Ex: $S_1 = \{1, 2\}$ $S_2 = \{1, 2, 3\}$

Subset => print(s1.issubset(s2)) => True ($S_1 <= S_2$)

Superset => print(s2.issuperset(s1)) => True ($S_2 >= S_1$)

Disjoint => (s1.isdisjoint({4,5})) => True
↳ True if they dont have common elements

Dictionary: Stores information in key-value pairs

→ Order, mutable, unique_keys,
↓
immutable data types

values → can be anything list, obj, other dictionaries too

Ex:

```
user1 = {
    "name": "hacker",
    "score": 95,
    "is_active": true}
```

```
user2 = dict(
    name="2",
    score=90,
    active=true)
```

access ↴
Print(user.get("email") => None

update or add ↴ user["email"] = "abc@gmail.com"

removing → user.pop("score")
user.popitem() → remove last pair as tuple

del user["is_active"]

user.clear() → clear entire dictionaries

Iterating
for user in user1: print(user) → keys
for value in user1.values(): print(value) → return values
for user, value in user1.items(): return both key and value